

Aula 05: Fundamentos JS e Funções

Descrição: Nesta aula, lançaremos as bases da lógica de programação em JavaScript. Vamos entender como armazenar dados, manipulá-los e, crucialmente, como encapsular lógica reutilizável através de funções para criar a nossa primeira biblioteca utilitária: `utils.js`.

PARTE 1: INTRODUÇÃO AO JAVASCRIPT E VARIÁVEIS

01: FUNDAMENTOS JS E FUNÇÕES

O Motor da Web Moderna

02: O QUE É JAVASCRIPT HOJE?

É uma linguagem de programação de alto nível, interpretada (ou compilada JIT), prototipada e multiparadigma. É a linguagem nativa dos navegadores e, com Node.js, domina também o back-end.

03: VARIÁVEIS: ONDE TUDO COMEÇA

Variáveis são 'caixas' ou contêineres rotulados para armazenar dados na memória do computador. No JS moderno, temos formas específicas de declarar essas caixas dependendo de como vamos usar o dado.

04: A EVOLUÇÃO: VAR VS LET VS CONST

- **var:** A forma antiga. Possui escopo de função e comportamento imprevisível (hoisting). Evite no JS moderno.
- **let:** Introduzido no ES6. Possui escopo de bloco e pode ter seu valor reatribuído.
- **const:** Também do ES6. Possui escopo de bloco, mas seu valor não pode ser reatribuído após a inicialização.

05: USO PRÁTICO: ESCOPO DE BLOCO

O escopo de bloco (`let/const`) impede que variáveis 'vazem' de dentro de estruturas como `if` ou `for`.

```
if (true) {  
  let local = 'visível apenas aqui';  
  var global = 'vaza para fora';  
}  
// console.log(local); // Erro  
console.log(global); // Funciona
```

06: A REGRA DE OURO: CONST POR PADRÃO

Sempre use `const` para declarar variáveis. Se você descobrir depois que precisa mudar o valor dela (como em um contador), altere para `let`. Isso previne bugs acidentais de reatribuição.

PARTE 2: TIPOS DE DADOS E OPERADORES

07: TIPOS DE DADOS DINÂMICOS

O JavaScript é fracamente tipado e dinâmico. Você não precisa declarar o tipo, e uma variável pode mudar de tipo durante a execução.

```
let dado = 'Olá'; // tipo String
dado = 42; // agora é Number
```

08: TIPOS PRIMITIVOS I: STRING E NUMBER

- **String:** Sequência de caracteres. Usamos aspas simples (' '), duplas (" ") ou crases (``) para Template Strings.
- **Number:** Inclui inteiros e números de ponto flutuante (decimais). O JS não faz distinção.

09: TEMPLATE STRINGS (CRASES)

A forma moderna e poderosa de trabalhar com texto, permitindo quebra de linha e interpolação de variáveis.

```
const nome = 'Maria';
const msg = `Olá, ${nome}.
Bem-vinda!`;
```

10: TIPOS PRIMITIVOS II: BOOLEAN E NULL/UNDEFINED

- **Boolean:** Apenas dois valores: true (verdadeiro) ou false (falso). Base para lógica de decisão.
- **Undefined:** Valor padrão de uma variável declarada mas não inicializada.
- **Null:** Representa a ausência intencional de qualquer valor de objeto.

11: ATIVIDADE I: PESQUISA E CURIOSIDADE

ATIVIDADE I: PESQUISA E CURIOSIDADE

=== *Trabalho em Grupo (3 Pessoas)* ===

- Pesquisem na documentação MDN (Mozilla Developer Network) e respondam:
- O que acontece se você usar o operador typeof em um valor null? Por que isso acontece?
- Quais são os 7 tipos primitivos do JavaScript no ES6+?
- Qual a diferença prática entre == (igualdade ampla) e === (igualdade estrita)?

12: OPERADORES ARITMÉTICOS

Símbolos para realizar cálculos matemáticos padrão.

+ (adição/concatenação), **-** (subtração), ***** (multiplicação), **/** (divisão), **%** (módulo/resto), ****** (exponenciação).

 **Uso Prático:** O operador % (módulo) é essencial para descobrir se um número é par (`n % 2 === 0`).

13: OPERADORES DE ATRIBUIÇÃO

Formas curtas de atualizar o valor de uma variável baseada em seu valor anterior.

```
let x = 10;  
x += 5; // x = x + 5 (resultado: 15)  
x *= 2; // x = x * 2 (resultado: 30)
```

14: OPERADORES DE COMPARAÇÃO

Usados em estruturas condicionais para retornar um Booleano.

=== (igual estrito), !== (diferente estrito), >, <, >=, <=.

 **Aviso: Sempre prefira === e !== para evitar coerção de tipo indesejada (ex: '5' == 5 é true, mas '5' === 5 é false).**

15: OPERADORES LÓGICOS

Permitem combinar múltiplas comparações.

- **&& (E):** Retorna true se AMBOS forem verdadeiros.
- **|| (OU):** Retorna true se PELO MENOS UM for verdadeiro.
- **! (NÃO):** Inverte o valor booleano.

16: USO PRÁTICO: LÓGICA COMBINADA

```
const idade = 20;  
const temCnh = true;  
const podeDirigir = idade >= 18 && temCnh;  
// resultado: true
```

PARTE 3: ESTRUTURAS DE CONTROLE E REPETIÇÃO

17: CONTROLE DE FLUXO: IF E ELSE

A estrutura if...else permite que o código tome decisões com base em condições lógicas avaliadas como Booleanas.

```
const hora = 14;  
if (hora < 12) {  
  console.log('Bom dia!');  
} else {  
  console.log('Boa tarde ou noite!');  
}
```

 **Uso Prático: É o mecanismo básico de bifurcação de caminhos no seu software.**

18: CONDIÇÕES MÚLTIPLAS: ELSE IF

Quando temos mais de duas possibilidades, podemos encadear os testes para cobrir cenários progressivos.

```
const nota = 85;
if (nota >= 90) {
  console.log('Conceito A');
} else if (nota >= 80) {
  console.log('Conceito B');
} else {
  console.log('Precisa melhorar');
}
```

 **Dica: A ordem importa! O JavaScript avalia de cima para baixo e executa apenas o primeiro bloco verdadeiro.**

19: OPERADOR TERNÁRIO

Uma forma concisa (short-hand) de escrever um if...else em uma única linha. É indispensável no desenvolvimento moderno.

```
// Sintaxe: condicao ? retorno_se_verdadeiro : retorno_se_falso
const idade = 20;
const status = idade >= 18 ? 'Maior de idade' : 'Menor de idade';
console.log(status); // Imprime: 'Maior de idade'
```

20: LAÇOS DE REPETIÇÃO (LOOPS)

Loops servem para executar o mesmo bloco de código repetidas vezes até que uma condição de parada seja alcançada. Isso evita a repetição manual do código e automatiza fluxos (conceito DRY - Don't Repeat Yourself).

- **for:** Excelente para quando você sabe a quantidade exata de vezes que precisa repetir.
- **while:** Excelente para quando o número de repetições é incerto, dependendo apenas de uma condição.

21: O CLÁSSICO: LAÇO FOR

O laço for tradicional possui 3 parâmetros definidos: a inicialização, a condição de parada, e a taxa de incremento.

```
// for (inicializacao; condicao; incremento)
for (let i = 0; i < 5; i++) {
  console.log(`Contador: ${i}`);
}
```

 **Aviso: Note o uso do let para inicializar o controle i , pois seu valor precisa ser constantemente reatribuído (i++) a cada nova volta no loop.**

22: O FLEXÍVEL: LAÇO WHILE

O while avalia uma condição, executando o bloco sempre que ela for true. Devemos garantir que o controle de estado avance para o final para não travarmos o código.

```
let energia = 100;
while (energia > 0) {
  console.log('Jogando...');
  energia -= 20; // Consome a energia até virar 0
}
console.log('Fim de jogo!');
```

23: A FORMA MODERNA: FOR...OF

Introduzido no ES6, é a maneira mais elegante e segura de iterar sobre estruturas de dados como arrays e strings (valores iteráveis).

```
const frutas = ['Maçã', 'Banana', 'Uva'];
for (const fruta of frutas) {
  console.log(fruta);
}
```

 **Por que é melhor? Evita erros clássicos de esquecer o controle numérico ou errar o índice. Podemos usar const pois uma nova variável independente é criada a cada loop!**

PARTE 4: FUNDAMENTOS DE FUNÇÕES

24: INTRODUÇÃO ÀS FUNÇÕES

Funções são blocos de código 'empacotados' projetados para executar uma tarefa específica. Elas são os blocos de construção fundamentais da reutilização de código.

25: ANATOMIA DE UMA FUNÇÃO (DECLARAÇÃO)

```
function nomeDaFuncao(parametro1, parametro2) {
  // Corpo da função (lógica)
  return resultado; // opcional
}
```

26: PARÂMETROS E ARGUMENTOS

- **Parâmetros:** Os nomes listados na definição da função (as variáveis locais).
- **Argumentos:** Os valores reais passados para a função quando ela é chamada (invocada).

27: A IMPORTÂNCIA DO 'RETURN'

A palavra-chave `return` finaliza a execução da função e envia um valor de volta para quem a chamou. Sem `return`, a função executa a lógica mas retorna `undefined`.

28: EXPRESSÕES DE FUNÇÃO

Em JS, funções são 'cidadãos de primeira classe'. Podem ser armazenadas em variáveis.

```
const subtrair = function(a, b) {  
  return a - b;  
};
```

29: ARROW FUNCTIONS (SETA)

A sintaxe moderna e curta introduzida no ES6. Muito usada em callbacks.

```
const multiplicar = (a, b) => {  
  return a * b;  
};
```

30: ARROW FUNCTIONS: RETORNO IMPLÍCITO

Se a função tiver apenas uma linha de lógica que retorna algo, podemos remover as chaves e o `return`.

```
const dobro = n => n * 2;
```

 **Uso Prático: Ideal para funções matemáticas simples ou manipulação rápida de dados.**

31: ATIVIDADE II: CRIANDO PEQUENAS LÓGICAS

ATIVIDADE II: CRIANDO PEQUENAS LÓGICAS

=== *Prática Individual* ===

- Escreva no papel ou editor de texto 3 funções usando a sintaxe de Arrow Function com retorno implícito:
- `isAdulto(idade)`: Retorna `true` se `idade >= 18`.
- `getAreaQuadrado(lado)`: Retorna a área.
- `formatarReal(valor)`: Recebe um número e retorna uma string formatada (ex: Recebe 5, retorna 'R\$ 5.00'). Dica: use `.toFixed(2)`

32: ESCOPO DE FUNÇÃO

Variáveis definidas dentro de uma função não podem ser acessadas de fora dela.

```
function teste() {  
  const interna = 'segredo';  
}  
  
console.log(interna); // Erro: interna is not defined
```

33: ESCOPO GLOBAL

Variáveis definidas fora de qualquer função ou bloco são globais e podem ser acessadas e modificadas por qualquer parte do script. Evite poluir o escopo global.

PARTE 5: FUNÇÕES UTILITÁRIAS, CALLBACKS E MÓDULOS (UTILS.JS)

34: INTRODUÇÃO ÀS FUNÇÕES UTILITÁRIAS

Uma função utilitária (utility function) é uma função projetada para realizar uma tarefa comum, genérica e repetitiva em um projeto. Elas não pertencem à lógica de negócio específica, mas ajudam a construí-la.

35: POR QUE CRIAR UMA UTILS.JS?

- **DRY (Don't Repeat Yourself):** Evita copiar e colar a mesma lógica (ex: formatação de data) em vários arquivos.
- **Manutenibilidade:** Se a lógica de formatação mudar, você muda em apenas um lugar.
- **Organização:** Mantém o código principal limpo e focado.

36: EXEMPLO ÚTIL: MASCARAR E-MAIL

Uma função que pega 'contato@email.com' e transforma em 'con***@email.com'.

 **Aplicação: Segurança e privacidade ao exibir dados parciais do usuário na interface.**

37: EXEMPLO ÚTIL: GERAR ID ÚNICO

```
const gerarId = () => {  
  return `id_${Date.now()}_${Math.random()}`;  
};
```

 **Aplicação: Essencial para criar novos itens em uma lista dinamicamente e garantir que cada um tenha uma referência única.**

38: EXEMPLO ÚTIL: CAPITALIZAR TEXTO

Transformar 'joão silva' em 'João Silva'. O JS não tem uma função nativa simples para isso.

39: CALLBACKS: INTRODUÇÃO

Um callback é uma função passada como argumento para outra função, para ser executada mais tarde.

```
setTimeout(() => {  
  console.log('Passou 1 segundo');  
}, 1000);
```

40: SETTIMEOUT E SETINTERVAL

- **setTimeout:** Executa a função callback UMA vez após um delay.

- **setInterval:** Executa a função callback repetidamente a cada intervalo de tempo.

41: PARÂMETROS PADRÃO (DEFAULT)

No ES6+, você pode definir valores padrão para parâmetros caso nenhum argumento seja passado.

```
function saudacao(nome = 'Visitante') {  
  console.log(`Olá, ${nome}`);  
}
```

42: BOAS PRÁTICAS EM FUNÇÕES

- **Nomes Descritivos:** Nomeie funções com verbos (ex: calcularTotal, não total).
- **Responsabilidade Única:** Uma função deve fazer APENAS UMA coisa (Princípio da Responsabilidade Única).
- **Legibilidade:** Mantenha as funções curtas e legíveis.
- **Sintaxe Moderna:** Use Arrow Functions para funções anônimas e callbacks.

43: O CONCEITO DE MÓDULO (PRÉVIA)

Para usar o utils.js em outros arquivos, precisaremos exportar as funções dele e importá-las onde for necessário. No navegador moderno, usamos export/import.

44: EXPORT E IMPORT (SINTAXE)

```
// No utils.js  
export const dobro = n => n * 2;  
  
// No main.js  
import { dobro } from './utils.js';
```

45: ATIVIDADE III: IMPLEMENTANDO A UTILS.JS

ATIVIDADE III: IMPLEMENTANDO A UTILS.JS

=== Desafio Prático Final ===

- Vamos criar a base da nossa biblioteca utilitária do projeto. Crie um arquivo utils.js e implemente (e exporte) as seguintes funções:
- **formatarMoeda(valor):** Recebe um número, retorna string 'R\$ X,XX' (use .toFixed(2) e troque '.' por ',').
- **validarEmail(email):** Recebe string, retorna true se tiver '@' e '.' (use .includes()).
- **obterDataFormatada():** Retorna a data atual no formato 'DD/MM/AAAA' (use objeto Date).

46: CHECKLIST DE ENTREGA

- ☐ O arquivo utils.js foi criado?
- ☐ As 3 funções foram implementadas corretamente?
- ☐ As funções estão usando const/let corretamente e sintaxe moderna (Arrow Functions onde couber)?
- ☐ Todas as funções foram exportadas usando export?

47: CONCLUSÃO

=== Base Sólida, Futuro Promissor ===

Dominar variáveis, tipos e funções é o passo mais importante no desenvolvimento web. Com a `utils.js`, você começa a pensar como um engenheiro de software: criando ferramentas reutilizáveis e organizadas.